



Langage de Programmation C

[plan]

1. Introduction
 - Concepts de bases
2. Histoire du langage C
3. Introduction au langage C
4. Éléments de base (règles d'écriture, types...)
5. Les variables
6. Opérateurs et expressions
7. Les entrées-Sorties (printf, scanf)

[Concepts de base]

Langages informatiques

- Un langage informatique est un outil permettant de donner des ordres (**instructions**) à la machine
 - A chaque instruction correspond une action du processeur
- Intérêt : écrire des **programmes (suite consécutive d'instructions)** destinés à effectuer une tâche donnée
 - Exemple: un programme de gestion de comptes bancaires
- Contrainte: être compréhensible par la machine

Concepts de base

Langage machine

- Langage binaire: l'information est exprimée et manipulée sous forme d'une suite de bits (Un bit: *binary digit*) = 0 ou 1
- Une combinaison de 8 bits= 1 Octet → $2^8 = 256$ possibilités qui permettent de coder tous les caractères alphabétiques, numériques, et symboles tels que ?, *, &, ...
- • Le code **ASCII** (*American Standard Code for Information Interchange*) donne les correspondances entre les caractères alphanumériques et leurs représentation binaire, Ex. A= 01000001, ?=00111111
- Les opérations logiques et arithmétiques de base (addition, multiplication, ...) sont effectuées en binaire

Concepts de base

L'assembleur

- Problème: le langage machine est difficile à comprendre par l'humain
- Idée: trouver un langage compréhensible par l'homme qui sera ensuite converti en langage machine .

- **Assembleur** : exprimer les instructions élémentaires de façon symbolique

```
ADD A, 4  
LOAD B  
MOV A, OUT  
...
```

traducteur → langage machine

- +: déjà plus accessible que le langage machine
- -: dépend du type de la machine (n'est pas **portable**)
- -: pas assez efficace pour développer des applications complexes

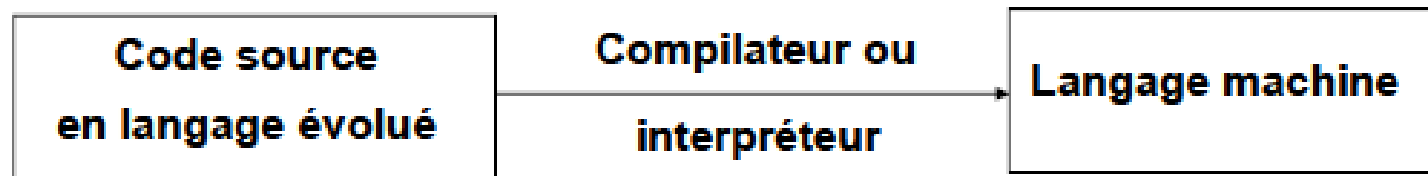


Apparition des langages évolués⁵

Concepts de base

Langages haut niveau

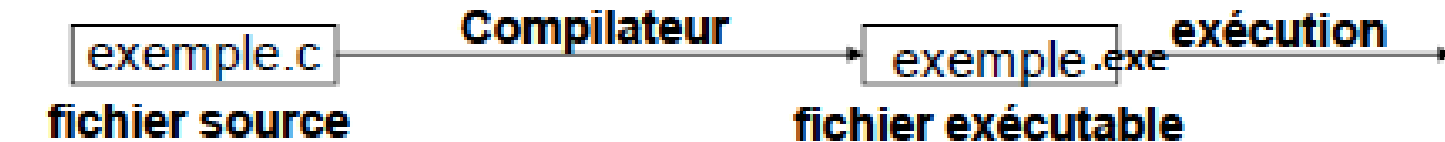
- Intérêts multiples pour le haut niveau:
 - proche du langage humain «anglais» (compréhensible)
 - permet une plus grande portabilité (indépendant du matériel)
 - Manipulation de données et d'expressions complexes (réels, objets, $a*b/c$, ...)
- Nécessité d'un traducteur (compilateur/interpréteur),



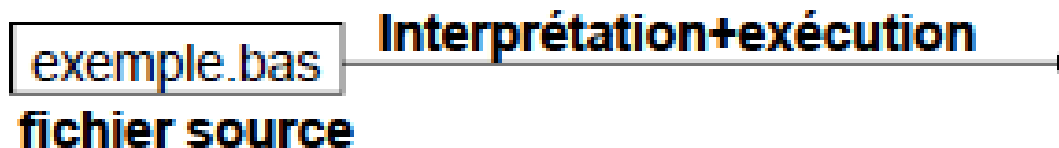
Concepts de base

Compilateur/interpréteur

- Compilateur: traduire le programme entier une fois pour toutes



- - + plus rapide à l'exécution
 - + sécurité du code source
 - - il faut recompiler à chaque modification
- Interpréteur: traduire au fur et à mesure les instructions du programme à chaque exécution
 - + exécution instantanée appréciable pour les débutants



[Concepts de base]

Langages de programmation:

- Deux types de langages:
 - Langages procéduraux
 - Langages orientés objets
- Exemples de langages:
 - **Fortran, Cobol, Pascal, C, ...**
 - **C++, Java, ...**

Historique du C

Le langage C a été conçu en 1972 dans «Bell Laboratories » par *Dennis Ritchie* avec l'objectif d'écrire un système d'exploitation (UNIX).

- Une 1^{ère} définition du langage est apparue en 1978 avec l'ouvrage de *Ritchie* et Kernighan « **The C programming language** ».

- Son succès international a contribué à sa normalisation:

- 1- **ANSI** (**A**merican **N**ational **S**tandard **I**nstitute

- 2- **ISO** (**I**nternational **S**tandardization **O**rganisation

- 3- **CEN** (**C**omité **E**uropéen de **N**ormalisation) en 1993

[Caractéristiques du langage C]

- **Universel** (général) : n'est pas orienté vers un domaine d'application particulier (applications scientifiques, de gestion, ...)
- **Près de la machine** : offre des opérateurs qui sont proches de ceux du langage machine (manipulations de bits, d'adresses, ...) → efficace
- **Modulaire**: peut être découpé en modules qui peuvent être compilés
Séparément
- **Portable**: en respectant le standard ANSI-C, il est possible d'utiliser le même programme sur plusieurs systèmes (hardware, système d'exploitation)

Introduction au langage C

Qu'est ce qu'un programme C?

C'est un texte écrit avec un éditeur de texte, respectant une certaine syntaxe et stocké sous forme d'un ou plusieurs fichiers (généralement avec l'extension .c). A l'opposé du langage assembleur, les instructions du langage C sont **obligatoirement** encapsulées dans des fonctions et il existe une fonction privilégiée appelée **main** qui est le point de départ de tout programme.

Exemple:

```
main()
{
    printf("Bonjour!");
}
```

Source.c

Compilation

Objet.o

Edition des liens

Exécutable

Introduction au langage C

les fichiers include

Exemple:

```
# include <stdio.h>
main()
{
    printf("Bonjour!");
}
```

La directive `#include` inclut le fichier `stdio.h` au programme avant la compilation (pour pouvoir utiliser la fonction prédéfinie `printf`).

On parle alors de préprocesseur

Introduction au langage C

les Commentaires

Exemple:

```
# include <stdio.h> //pour pouvoir utiliser la fonction printf
main()
{
printf("Bonjour!");
}
/*Ce programme imprime la chaine de
caractère "Bonjour!" à l'écran*/
```

Introduction au langage C

Présentation de quelques instructions du langage C

La fonction **main** est la fonction principale des programmes en C:

```
#include <stdio.h>
#include <math.h>
#define NFOIS 5

main()
```

Déclaration de variables

```
{
    int i;
    float x;
    float racinex;
```

• **printf()** écriture formatée de données

```
    printf("Bonjour!\n");
    printf("Je vais vous calculer %d racines carrées\n", NFOIS);
```

• **scanf()** lecture formatée de données

```
    for(i=0; i<NFOIS; i++)
    {
        printf("Donnez un nombre: ");
        scanf("%f", &x);
        if (x<0)
            printf("Le nombre %f ne possède pas de racine carrée\n", x);
        else
        {
            racinex=sqrt(x);
            printf("Le nombre %f a pour racine carrée : %f\n", x, racinex);
        }
    }
```

```
    printf("Travail terminée - Au revoir!");
}
```

• Directive du préprocesseur

- inclusion des fichiers d'en-tête (fichiers avec extension **.h**)
- définitions des constantes avec **#define**

Introduction au langage C

Présentation de quelques instructions du langage C

Exemple 2 :

```
#include <stdio.h>

main()
{
    char op;
    int n1,n2;

    printf("opération souhaitée (+ ou *) ?");
    scanf("%c", &op);

    printf("Donnez 2 nombres entiers :");
    scanf("%d %d", &n1, &n2);

    if (op=='+' ) printf("leur somme est : %d", n1+n2);
        else printf("leur produit est : %d", n1*n2);
}
```

[Éléments de base]

***Variables, types, opérateurs
et
expressions***

[Les variables]

- Les variables servent à stocker les valeurs des données utilisées pendant l'exécution d'un programme
- Les variables doivent être **déclarées avant d'être** utilisées, elles doivent être caractérisées par :
 - un nom (**Identificateur**)
 - un **type** (**entier, réel, ...**)

(Les types de variables en C seront discutés par la suite)

Les identificateurs

- Le choix d'un identificateur (nom d'une variable ou d'une fonction) est soumis à quelques règles :

- doit être constitué uniquement de lettres, de chiffres et du caractère souligné _ (Eviter les caractères de ponctuation et les espaces)

correct: `PRIX_HT, prixHT`

incorrect: `PRIX-HT, prix HT, prix.HT`

- doit commencer par une lettre (y compris le caractère souligné)

correct : `A1, _A1`

incorrect: `1A`

- doit être différent des mots réservés du langage :

**auto break case char const continue default do double else enum
extern float for goto if int long register return short signed sizeof
static struct switch typedef union unsigned void volatile while**

Remarque : C distingue les majuscules et les minuscules. `NOMBRE` et `nombre` sont des identificateurs différents

[Les Types]

- Le type d'une variable détermine l'ensemble des valeurs qu'elle peut prendre et le nombre d'octets à lui réserver en mémoire

Les types de base du langage C se répartissent en 3 grande catégories en fonction de la nature des informations qu'ils permettent de représenter:

- Nombres **entiers** (*int*)
- Nombres **flottants** (*float* ou *double*)
- **Caractère** (*char*): apparaît en C comme un cas particulier de *int*.

Ils peuvent être signés ou non signés : *signed* ou *unsigned* (*unsigned int x;*)

Les Types

Remarques:

- Un type de base est un type pour lequel une variable peut prendre une seule valeur à un instant donné contrairement aux types agrégés
- Le type caractère apparaît en C comme cas particulier du type entier (un caractère est un nombre entier, il s'identifie à son code ASCII)
 - Le code **ASCII** (*American Standard Code for Information Interchange*) donne les correspondances entre les caractères alphanumériques et leurs représentation binaire, Ex. A= 01000001, ?=00111111
- En C il n'existe pas de type spécial pour chaînes de caractères. Les moyens de traiter les chaînes de caractères seront présentés aux chapitres suivants
- Le type booléen n'existe pas. Un booléen est représenté par un entier (un entier non nul équivaut à vrai et la valeur zero équivaut à faux)

Les Types

a. Les Types Entiers (2 ou 4 octets)

- **int** (entier sur 32 bits : - 2 147 483 648 à 2 147 483 647)

Format d'affichage: %d.

Les types dérivés

short int ou **short** (entier sur 16 bits (2 octets) : - 32 768 à 32 767)

Format d'affichage: %d (ou %hd)

long int ou **long** (entier sur 32 bits ou 64 bits, selon les machines)

Format d'affichage: %ld

unsigned int : entier non signé (sur 2 ou 4 octets)

Format d'affichage: %u

unsigned short : 0 à 65535

unsigned long : entier long positif 0 à 4294967295

Format d'affichage: %lu

pas de bit réservé pour le signe

- C accepte les constantes entière en notation décimale, hexadécimale ou octale²

Les Types dans C

b. Le Type Char (1 octet)

- En C, un caractère est un entier signé codé sur 1 octet
- **char : caractères (entier sur 1 octet : - 128 à 127)**
- Remarque : Une variable du type **char** peut subir les mêmes opérations que les variables du type **short, int ou long**

```
main()
{
    char c = 'A';
    printf("%c", c + 1);
}
```

l'expression **c+1** est valide. Elle désigne le caractère suivant dans le code ASCII. Ainsi, le programme suivant imprime le caractère 'B'.

- **Notation des constantes caractères :** 'a', '\$' ..

Il existe des caractères non imprimables, tel que le changement de ligne, de Tabulation, ... en voici un tableau récapitulatif

Les Types dans C

NOTATION	RESULTAT
\a	cloche ou bip (alert ou audible bell)
\b	Retour arrière (Backspace)
\f	Saut de page (Form Feed)
\n	Saut de Ligne (Line Feed)
\r	Retour chariot (Carriage Return)
\t	Tabulation horizontaLe (Horizontal Tab)
\v	Tabutation verticale (Vertical Tab)
\\	\
\'	'
\"	"
\?	?

[Les Types dans C]

c. Les Types Flottants

- **Float** : réel simple précision codé sur 4 octets avec 1 bit de signe, 23 bits de mantisse et 8 bits d'exposant $[-3.4 \cdot 10^{38} .. 3.4 \cdot 10^{38}]$
- **Double** : réel double précision codé sur 8 octets avec 1 bit de signe, 52 bits de mantisse et 11 bits d'exposant $[-1.7 \cdot 10^{308} .. 1.7 \cdot 10^{308}]$
- **Long** : réel très grande précision codé sur 10 octets avec 1 bit de signe, 64 bits de mantisse et 15 bits d'exposant $[-3.4 \cdot 10^{4932} .. 3.4 \cdot 10^{4932}]$

[Les Types dans C]

A propos du type booléen :

Pas de type booléen en C. Le type booléen est représenté par un entier. Il se comporte comme la valeur booléenne vraie si cette valeur entière est non nulle.

valeur logique <u>f</u> aux	$\Leftrightarrow$	valeur numérique zéro
valeur logique <u>v</u> rai	$\Leftrightarrow$	toute valeur différente de zéro

Dans un contexte qui exige une valeur booléenne (comme les tests, par exemple), un entier non nul équivaut à vrai et la valeur zero équivaut à faux

Déclaration de variables

- Les *déclarations* introduisent les variables qui seront utilisées, fixent leur type et parfois aussi leur valeur de départ (initialisation)

- Syntaxe de déclaration en C

<Type> <NomVar1>,<NomVar2>,...,<NomVarN>;

- Exemple:

int i, j,k;

float x, y ;

double z=1.5; // déclaration et initialisation

short compteur;

char c=`A`;

Déclaration de constantes

- Une constante conserve sa valeur pendant toute l'exécution d'un programme
- En C, on associe une valeur à une constante en utilisant :

- **la directive *#define* :**

#define nom_constant valeur

Ici la constante ne possède pas de type.

exemple: *#define Pi 3.141592*

- **le mot clé *const* :**

const type nom = expression ;

Dans cette instruction la constante est typée

exemple : *const float Pi =3.141592*

(Rq: L'intérêt des constantes est de donner un nom parlant à une valeur, par exemple NB_LIGNES, aussi ça facilite la modification du code)

Constantes entières

On distingue 3 formes de constantes entières :

- **forme décimale** : *c'est l'écriture usuelle. Ex : 372, 200*
- **forme octale** (base 8) : on commence par un 0 suivi de chiffres octaux. Ex : 0477
- **forme hexadécimale** (base 16) : on commence par 0x (ou 0X) suivis de chiffres hexadécimaux (0-9 a-f). Ex : 0x5a2b, 0Xa9f

Constantes entières

Remarques

- Le compilateur attribue automatiquement un type aux constantes entières. Il attribue en général le type le plus **économique** parmi (int,unsigned int, long int, unsigned long int)
- On peut forcer la machine à utiliser un type de notre choix en ajoutant les suffixes suivants:
 - **u** ou **U** pour **unsigned int**, **Ex** : 100**U**, 0xA**u**
 - **l** ou **L** pour **long**, **Ex** : 15**l**, 0127**L**
 - **ul** ou **UL** pour **unsigned long**, **Ex** : 1236**UL**, 035**ul**

Les expressions et les instructions

Expressions

La formation des expressions est définie par récurrence :

- Les *constantes* et les *variables* sont des expressions.
- Les expressions peuvent être combinées entre elles par des *opérateurs* et former ainsi des expressions plus complexes.
- Les expressions peuvent contenir des appels de fonctions et elles peuvent apparaître comme paramètres dans des appels de *fonctions*.

Exemples

```
i =0
```

```
i++
```

```
X=pow(A,4)
```

```
printf(" Bonjour !\n")
```

```
a=(5*x+10*y)*2
```

```
(a+b)>=100
```

Les expressions et les instructions

Instructions

- Une expression comme `i=0` ou `i++` ou `printf(...)` devient une *instruction*, si elle est suivie d'un point-virgule.

Exemples

```
i=0;  
i++;  
X=pow(A,4);  
printf(" Bonjour !\n");  
a=(5*x+10*y)*2;
```

Évaluation et résultats

- Une expression fournit une seule valeur, elle est évaluée en respectant des règles de priorité et d'associativité (seront vues dans la suite du cours)
 - `(3+2==5)` retourne la valeur 1 (vrai)
 - `A=5+3` retourne la valeur 8

Opérateurs en C

- On distingue les opérateurs suivants en C :
 - les opérateurs arithmétiques : **+**, **-**, *****, **/**, **%**
 - les opérateurs d'affectation : **=**, **+=**, **-=**, ***=**, **/=**, ...
 - les opérateurs logiques : **&&**, **||**, **!**
 - les opérateurs Relationnels : **==**, **!=**, **<**, **>**, **<=**, **>=**
 - les opérateurs d'incrémentation et de décrémentation : **++**, **--**
 - les opérateurs sur les bits : **<<**, **>>**, **&**, **|**, **~**, **^**
 - d'autres opérateurs particuliers : **?:**, **sizeof**, **cast**

Opérateurs arithmétiques

- C dispose d'opérateurs classiques **binaires** : (+), (-), (*) et la division (/), ainsi que d'un opérateur "**unaire**" correspondant à l'opposé noté (-) (comme dans -n ou -x+y).
- Les opérands peuvent être des entiers ou des réels sauf pour % qui agit uniquement sur des entiers et fournit le reste de la division de son premier opérande par son second. Par **exemple**, 11%4 vaut 3, 23%6 vaut 5
- **Remarque**:
 - le quotient de **deux entiers** fournit un **entier**. Ainsi 5/2 vaut 2;
 - le quotient de deux flottants (noté, lui aussi, /) est bien un flottant (5.0/2.0 vaut bien approximativement 2.5).
 - Il retourne aussi un quotient flottant si l'un au moins des opérands est un **flottant** (5.0 / 2 vaut 2.5)

Priorité des opérateurs :

les règles sont "naturelles" et rejoignent celles de l'algèbre traditionnelle

Opérateurs arithmétiques

Conversions implicites

- Les types `short` et `char` sont systématiquement convertis en `int` indépendamment des autres opérandes
- La conversion se fait en général selon une hiérarchie qui n'altère pas les valeurs
`int` → `long` → `float` → `double` → `long double`
- **Exemple1 :** `n * x + p` (`int n,p; float x`)
 - exécution prioritaire de `n * x` : conversion de `n` en `float`
 - exécution de l'addition : conversion de `p` en `float`
- **Exemple2 :** `p1 * p2 + p3 * x` (`char p1, short p2, p3 ; float x`)
 - `p1`, `p2` et `p3` d'abord convertis en `int`
 - `p3` converti en `float` avant multiplication

Opérateurs arithmétiques

Exemple de conversion

Exemple : $n * p + x$ (int n ; long p ; float x)

$n * p + x$

long



*



long



float



+



float

conversion de n en long

multiplication par p

$n * p$ de type long

conversion de $n * p$ en float

addition

résultat de type float

Opérateur d'affectation ordinaire

- L'opérateur = affecte une valeur ou une expression à une variable
Exemple: `double x,y,z; x=2.5; y=0.7; z=x*y-3;`
- Le terme à gauche de l'affectation est appelé *lvalue* (*left value*)
- L'affectation est interprétée comme une expression. La valeur de l'expression est la valeur affectée
- On peut enchaîner des affectations, l'évaluation se fait de droite à gauche
exemple : `i = j = k = 5` (est équivalente à `k = 5`, `j=k` et ensuite `i=j`)
- La valeur affectée est toujours convertie dans le type de la *lvalue*, même si ce type est plus faible (ex : conversion de `float` en `int`, avec perte d'information)

Comme les affectations sont aussi interprétées comme des expressions, il est possible de profiter de la valeur rendue par l'affectation:

➤ `((A=sin(X)) == 0.5)`

Opérateurs Relationnels

- $<$: inférieur à $<=$: inférieur ou égal à
- $>$: supérieur à $>=$: supérieur ou égal à
- $==$: égal à $!=$: différent de

- Le résultat de la comparaison n'est pas une valeur booléenne, mais
 - **0** si le résultat de la comparaison est **faux**,
 - **1** si le résultat de la comparaison est **vrai**.
- Les expressions relationnelles peuvent donc intervenir dans des expressions arithmétiques

Exemple: $a=2$, $b=7$, $c=4$

- $b==3 \rightarrow 0$ (faux)
- $a!=b \rightarrow 1$ (vrai)
- $4*(a<b) + 2*(c>=b) \rightarrow 4$

Opérateurs Relationnels

les expressions comparées pourront être de type quelconque et seront soumises aux règles de conversion présentées précédemment.

Liste des opérateurs:

$<$	$<=$	$>$	$>=$		$==$	$!=$
$\underbrace{\hspace{1.5cm}}$				$>$	$\underbrace{\hspace{1.5cm}}$	
même priorité					même priorité	

Exemple: $a < b == c < d \rightarrow (a < b) == (c < d)$

Remarque: les opérateurs relationnels sont moins prioritaires que les opérateurs arithmétiques

(	$x+y < a+2$	$\rightarrow$	$(x+y) < (a+2)$	)
---	-------------	---------------	-----------------	---

Opérateurs Logiques

- C dispose de trois opérateurs logiques classiques:
 - **et** (noté **&&**),
 - **ou** (noté **||**)
 - **non** (noté **!**).
- Par exemple: $(a < b) \ \&\& \ (c < d)$ prend la valeur 1 (vrai) si les deux expressions $a < b$ et $c < d$ sont toutes deux vraies (de valeur 1), la valeur 0 (faux) dans le cas contraire.
- Les valeurs numériques sont acceptées :
 - toute valeur non nulle correspond à vrai
 - 0 correspond à faux,

Exemple :

$5 \ \&\& \ 11 \rightarrow 1$

$!13.7 \rightarrow 0$

Opérateurs Logiques

Exemples:

n et p sont des entiers, le compilateur accepte des expressions telles que:

$n \ \&\& \ p$ $n \ || \ p$ $\text{if} (!n) \leftrightarrow \text{if} (n == 0)$

Remarque1: l'opérateur ! a une priorité supérieure à celle de tous les opérateurs arithmétiques et relationnels. La négation de $a==b$ serait **!(a==b)** et non **!a==b**

Remarque2: l'opérateur || est moins prioritaire que &&. Tous deux sont de priorité inférieure aux opérateurs arithmétique et relationnels

$a < b \ \&\& \ c < d$ équivaut à $(a < b) \ \&\& \ (c < d)$

Opérateurs Logiques

Remarque3:

Le 2ème opérande est évalué uniquement en cas de nécessité:

➤ **a && b** : b évalué uniquement si a vaut vrai (si a vaut faux, évaluation de b inutile car a && b vaut faux)

➤ **a || b** : **b évalué uniquement si a vaut faux (si a vaut vrai, évaluation de b inutile car a || b vaut vrai)**

Exemples

➤ if ((d != 0) && (n / d == 2)) : pas de division si d vaut 0

➤ if ((n >= 0) && (sqrt(n) < p)) : racine non calculée si n < 0

L'intérêt est d'accélérer l'évaluation et d'éviter les traitements inappropriés

Opérateurs d'Incrémentation et décrémentation

- Les opérateurs ++ et -- sont des opérateurs unaires permettant respectivement d'ajouter et de retrancher 1 au contenu de leur opérande
- Cette opération est effectuée après ou avant l'évaluation de l'expression suivant que l'opérateur suit ou précède son opérande
 - $k = i++$ (*post-incrémentation*) affecte d'abord la valeur de i à k et incrémente après ($k = i++ ; \quad k = i ; i = i+1 ;$)

 - $k = ++i$ (*pré-incrémentation*) incrémente d'abord et après affecte la valeur incrémentée à k ($k = ++i ; \quad i = i+1 ; k = i ;$)


Opérateurs d'Incrémentation et décrémentation

Ex. 1

```
int i=5,j,k;  
j=i++; /* j=5 puis i=6 */  
k=++i; /* i=7 puis k=7 */
```

Ex. 2

```
int i, x, t[5];  
i=3;  
x=t[i++]; /* x=t[i]=t[3] et i=i+1=3+1=4 */
```

Ex. 3

```
int i = 3;  
/* incrémentation de i */  
i++; /* i=4 */  
/* incrémentation de i */  
++i; /* i =5 */
```

Ex. 4

```
int i, x, t[5];  
i=3;  
x=t[++i]; /* i=i+1=3+1=4 et x=t[i]=t[4] */
```

Ex.5

$i = 5; n = ++i - 5;$

$i = 5; n = i++ - 5;$

i vaut 6 et n vaut 1

i vaut 6 et n vaut 0

- Remarque : idem pour l'opérateur de décrémentation --

Opérateurs d'Incrémentation et décrémentation

Priorité:

$3 * i++ * j-- + k++$ équivalent à $3 * (i++) * (j--) + (k++)$

La priorité élevée de ces opérateurs unaires permet d'écrire des expressions assez compliquées sans qu'il soit nécessaire d'employer des ()

Opérateurs de manipulations de bits

opérateurs arithmétiques bit à bit :

& : ET logique | : OU logique ^ : OU exclusif ~ : négation

Les opérandes sont de type entier. Les opérations s'effectuent bit à bit suivant la logique binaire

b1	b2	~b1	b1&b2	b1 b2	b1^b2
1	1	0	1	1	0
1	0	0	0	1	1
0	1	1	0	1	1
0	0	1	0	0	0

Ex : 14= 1110 , 9=1001 $\rightarrow$ 14 & 9= 1000=8, 14 | 9 =1111=15

Opérateurs de décalage de bits

Il existe **deux** opérateurs de décalage :

>> : décalage à droite

<< : décalage à gauche

L'opérande gauche constitue l'objet à décaler et l'opérande droit le nombre de bits de décalage.

Dans le cas d'un décalage à gauche les bits les plus à gauche sont perdus. Les positions binaires rendues vacantes sont remplies par des 0

Ex : char x=14; (14=00001110) $\longrightarrow$ $14 \ll 2 = 00111000 = 56$
char y=-7; (-7=11111001) $\longrightarrow$ $-7 \ll 2 = 11100100 = -28$

Rq : un décalage à gauche de k bits correspond (sauf débordement) à la multiplication par 2^k

Opérateurs de décalage de bits

Lors d'un décalage à droite les bits les plus à droite sont perdus.

- si l'entier à décaler est non signé, les positions binaires rendues vacantes sont remplies par des 0
- s'il est signé le remplissage dépend de l'implémentation (en général le remplissage se fait par le bit du signe)

Ex : char x=14; (14=00001110) $\longrightarrow$ 14>>2 = 00000011 = 3

Remarque : un décalage à droite ($n \gg k$) correspond à la division entière par 2^k si n est non signé

Opérateurs d'affectation combinés

Soit un opérateur de calcul *op* et deux expressions *exp1* et *exp2*.
L'expression *exp1 = exp1 op exp2* peut s'écrire en général de façon équivalente sous la forme *exp1 op = exp2*

Opérateurs utilisables :

+=	-=	*=	/=	%=
<<=	>>=	&=	^=	 =

Exemples :

- *a=a+b* s'écrit : *a+=b*
- *n=n%2* s'écrit : *n%=2*
- *x=x*i* s'écrit : *x*=i*
- *p=p>> 3* s'écrit : *p>>=3*

Opérateur de forçage de type (**cast**)

Il est possible d'effectuer des conversions explicites ou de forcer le type d'une expression

- Syntaxe : `(<type>) <expression>`
- **Exemple** : `int n, p ;`
`(double) (n / p);` convertit l'entier `n / p` en double

Remarque : la conversion (ou casting) se fait après calcul
`(double) (n/p) ≠ (double) n / p ≠ (double) (n) / (double) (p)`

Exemple

```
float n = 4.6, p = 1.5 ;  
(int) n / (int) p   = 4 / 1 = 4  
(int) n / p        = 4 / 1.5 = 2.66  
n / (int) p        = 4.6 / 1 = 4.6  
n / p            = 4.6 / 1.5 = 3.06
```

L'opérateur conditionnel ? :

(seul opérateur ternaire en C)

Syntaxe:

exp1 ? exp2 : exp3

exp1 est évaluée, si sa valeur est non nulle(vrai) c'est **exp2** qui est exécutée, sinon **exp3** (valeur de exp1 (faux))

exemples

➤ **max = a > b ? a : b**

Si a>b alors on affecte à max le contenu de exp2 càd a sinon on lui affecte b

➤ **z = (x=y) ? a : b ;**

➤ **a>b ? i++ : i-- ;**

utilisée

on utilise la valeur de l'expression
la valeur de l'expression n'est pas

[Opérateur séquentiel ,]

- **Utilité** : regrouper plusieurs sous-expressions ou calculs en une seule expression
- Les calculs sont évalués en séquence de gauche à droite
- La valeur de l'expression est celle de la dernière sous-expression

Exemples

- `i++ , i + j`; // on évalue `i++` ensuite `i+j` (on utilise la valeur de `i` incrémentée)
- `i++ , j = i + k , a + b`; // la valeur de l'expression est celle de `a+b`
- `for (i=1 , k=0 ; ... ; ...) { }`

[Opérateur SIZEOF]

Syntaxe : `sizeof (<type>)` ou `sizeof (<variable>)`
fournit la taille en octets d'un type ou d'une variable

Exemples

- `int n;`
- `printf ("%d \n",sizeof(int)); // affiche 4`
- `printf ("%d \n",sizeof(n)); // affiche 4`

Priorités des Opérateurs

Classe d'opérateur	Opérateur(s)	Associativité
Parenthésage	()	de gauche à droite
Suffixes	[] -> . ++ --	de gauche à droite
Unaires	& * + - ! sizeof ~	de droite à gauche
Changement de type	(type)	de droite à gauche
Multiplicatifs	* / %	de gauche à droite
Additifs	+ -	de gauche à droite
Décalage	<< >>	de gauche à droite
Comparaisons	< <= > >=	de gauche à droite
Égalités	== !=	de gauche à droite
ET bit à bit	&	de gauche à droite
OU exclusif bit à bit	^	de gauche à droite
OU bit à bit		de gauche à droite
ET logique	&&	de gauche à droite
OU logique		de gauche à droite
Condition	? :	de droite à gauche
Affectations	= += -= *= /= &= = ^= <<= >>=	de gauche à droite
Succession	,	de gauche à droite

Les Entrées-Sorties Conversationnelles

- Il s'agit des instructions permettant à la machine de dialoguer avec l'utilisateur
 - Dans un sens la **lecture** permet à l'utilisateur d'entrer des valeurs au clavier pour qu'elles soient utilisées par le programme
 - Dans l'autre sens, **l'écriture** permet au programme de communiquer des valeurs à l'utilisateur en les affichant à l'écran (ou en les écrivant dans un fichier)
- La bibliothèque standard **<stdio>** contient un ensemble de fonctions qui assurent la lecture et l'écriture des données.
-
- Dans ce chapitre, nous allons en discuter les plus importantes:
 - **printf()** écriture formatée de données
 - **scanf()** lecture formatée de données

La fonction *printf*

- la fonction **printf** est utilisée pour afficher à l'écran du texte, des valeurs de variables ou des résultats d'expressions.
- Syntaxe : **printf("format", expr1, expr2, ...);**
- **expr1,...** : sont les variables et les expressions dont les valeurs sont à représenter
- **Format** : est une chaîne de caractères qui peut contenir
 - du texte(des caractères à afficher tels quels)
 - des séquences d'échappement ('\\n', '\\t', ...)
 - des spécificateurs de format : un ou deux caractères précédés du symbole **%**, *indiquant le format d'affichage*

Exemple: printf("leur somme est : %d",n1+n2);

Rq : Le nombre de spécificateurs de format doit être égale au nombre d'expressions!

Spécificateurs de format

SYMBOLE	TYPE	AFFICHAGE COMME
<i>%d ou %i</i>	<i>int</i>	entier relatif
<i>%u</i>	<i>unsigned int</i>	entier naturel non signé
<i>%c</i>	<i>char</i>	caractère
<i>%o</i>	<i>int</i>	entier sous forme octale
<i>%x ou %X</i>	<i>int</i>	entier sous forme hexadécimale
<i>%f</i>	<i>float, double</i>	réel en notation décimale
<i>%e ou %E</i>	<i>float, double</i>	réel en notation exponentielle
<i>%s</i>	<i>char*</i>	chaîne de caractères



Formats d'impression pour la fonction printf

Format	conversion en	écriture
%d	int	décimale signée
%ld	long int	décimale signée
%u	unsigned int	décimale non signée
%lu	unsigned long int	décimale non signée
%o	unsigned int	octale non signée
%lo	unsigned long int	octale non signée
%x	unsigned int	hexadécimale non signée
%lx	unsigned long int	hexadécimale non signée
%f	double	décimale virgule fixe
%lf	long double	décimale virgule fixe
%e	double	décimale notation exponentielle
%le	long double	décimale notation exponentielle
%g	double	décimale, représentation la plus courte parmi %f et %e
%lg	long double	décimale, représentation la plus courte parmi %lf et %le
%c	unsigned char	caractère
%s	char*	chaîne de caractères

[Spécificateurs de format]

- **c** : char: caractère affiché "en clair" (convient aussi à short ou à int compte tenu des conversions systématiques)
- **d** : int (convient aussi à char, compte tenu des conversions systématiques)
- **u** : unsigned int (convient aussi à unsigned char ou à unsigned short, compte tenu des conversions systématiques)
- **ld** : long
- **lu** : unsigned long

Spécificateurs de format

- **f** : double ou float écrit en notation "décimale" avec six chiffres après le point
- **e** : double ou float écrit en notation "exponentielle" (mantisse entre 1 et 9) avec six chiffres après le point décimal, sous la forme $x.xxxxxxe+yyy$ ou $x.xxxxxxe-yyy$ pour les nombres positifs et $-x.xxxxxxe+yyy$ ou $-x.xxxxxxe-yyy$ pour les nombres négatifs
- **s** : chaîne de caractères dont on fournit l'adresse (notion qui sera étudiée ultérieurement)

Les possibilités de la fonction printf

Exemple 1

```
#include<stdio.h>
main()
{ int i=1 , j=2, N=15;
  printf("la somme de %d et %d est %d \n", i, j, i+j);
  printf(" N= %x \n" , N);
  char c='A' ;
  printf(" le code Ascii de %c est %d \n", c, c);
}
```

- Ce programme va afficher :
la somme de 1 et 2 est 3
N=f
le code Ascii de A est 65
- **Remarque** : Pour pouvoir traiter correctement les arguments du type long, il faut utiliser les spécificateurs %ld, %li, %lu, %lo, %lx

Les possibilités de la fonction printf

■ Exemple 2

```
#include<stdio.h>
main()
{ double x=10.5, y=2.5;
  printf("%f divisé par %f égal à %f \n", x, y, x/y);
  printf("%e divisé par %e égal à %e\n", x, y, x/y);
}
```

■ Ce programme va afficher :

10.500000 divisé par 2.500000 égal à 4.200000

1.050000e+001 divisé par 2.500000e+000 égal à 4.200000e+000

- **Remarque** : Pour pouvoir traiter correctement les arguments du type long double, il faut utiliser les spécificateurs %Lf et %le

Les possibilités de la fonction *printf*

Action sur le gabarit d'affichage:

Les entiers sont affichés par défaut sans espaces avant ou après. Les flottants avec six chiffres après le point.

Pour agir sur l'affichage → un nombre est placé après % et précise le nombre de caractère **minimum à utiliser**.

Exemples:

■ `printf("%3d", n);`

n = 20 ~20

n = 3 ~~3

n = 2358 2358

n = -5200 -5200

■ `printf("%4X", 123);` ~~7B

■ `printf("%4x", 123);` ~~7b

Les possibilités de la fonction *printf*

Action sur le gabarit d'affichage:

- Pour les réels(floatants), on peut préciser la *largeur minimale de la valeur à afficher* et le nombre de chiffres après le point décimal.
- La précision par défaut est fixée à six décimales. Les positions décimales sont arrondies à la valeur la plus proche.

Exemples:

```
printf("%f" , x );
```

➤ x = 1.2345 1.234500

➤ x = 12.3456789 12.345679

```
printf("%10f" , x );
```

➤ x = 1.2345 ~~1.234500

➤ x = 1.2345E5 123450.000000

```
printf("%e" , x );
```

➤ x = 1.2345 1.234500e+000

➤ x = 123.45 1.234500e+002

Les possibilités de la fonction *printf*

Actions sur la précision :

pour les flottants, on peut préciser un nombre de chiffres après le point décimal

Exemples:

```
printf("%10.3f" , x );
```

x = 1.2345 ~~~~~1.235

x = 1.2345E3 ~~1234.500

x = 1.2345E7 12345000.000

Remarques:

Cadrage de l'affichage à gauche → printf("%-10.3f" , x); x = 1.2345 → 1.235~~~~~

- Le caractère * figurants à la place d'un gabarit ou une précision signifie que la valeur effective est fournie dans la liste des arguments de *printf*:

```
printf("%8.*f" , n, x );    n=1; x=1.2345    → ~~~~~1.2
```

- La fonction *printf* fournit une **valeur de retour** (nombre de caractère affichés)

Les possibilités de la fonction printf

Les erreurs de programmation :

Erreur 1 : Code de format en désaccord avec le type de l'expression à afficher:

Conséquence : mauvaise interprétation si même taille (int en %u), sinon, conséquences plus désastreuses.

Erreur 2: Nombre de code de format différents du nombre d'expressions de la liste

Conséquence 1 : si des expressions de la liste n'ont pas de format, elles ne seront pas affichées (`printf("%d" , n, p );` // valeur de p ne sera pas affiché)

Conséquence 2 : s'il y a trop de code de format, printf cherchera à afficher n'importe quoi (`printf("%d %d" , n );` `n=8` → 8 2)

[La fonction scanf]

- la fonction **scanf** permet de lire des données à partir du clavier
- **Syntaxe** : **scanf("format", AdrVar1, AdrVar2, ...);**
 - **Format** : le format de lecture de données, est le même que pour *printf*
 - ***adrVar1, adrVar2, ...*** : **adresses des variables auxquelles les** données seront attribuées. L'adresse d'une variable est indiquée par le **nom de la variable précédé du signe &**

[Exemples de scanf]

Exemple1

```
#include<stdio.h>
main()
{ int i , j;
  scanf("%d%d", &i, &j);
  printf("i=%d et j=%d", i, j);
}
```

ce programme permet de lire deux entiers entrés au clavier et les afficher à l'écran.

Remarque : pour lire une donnée du type **long**, il faut utiliser les spécificateurs **%ld**, **%li**, **%lu**, **%lo**, **%lx**.

Possibilités de la fonction scanf()

Les possibilités de la fonction scanf

a. Les principaux codes de conversion :

- **f** ou **e** : float écrit en notation "décimale" ou "exponentielle"
- **Lf** ou **le** : double écrit en notation "décimale" ou "exponentielle"
- **s** : chaîne de caractères dont on fournit l'adresse (notion qui sera étudiée ultérieurement)

b. Premières notion de tampon et de séparateurs :

Lorsque **scanf** attend des données, l'information frappée au clavier est rangée temporairement dans l'emplacement mémoire nommé « **tampon** ». Ce dernier est exploré caractère par caractère au fur et à mesure des besoins. Certains caractères jouent un rôle particulier: les **séparateurs** (l'espace et la fin de ligne \n)

Les possibilités de la fonction scanf

c. Les premières règles utilisées par scanf :

- Les codes de format correspondant à un nombre entraînent l'avancement du pointeur jusqu'au 1^{er} caractère différent d'un séparateur, puis scanf prend en compte tous les caractères suivants jusqu'à la rencontre d'un séparateur.
- Quand au code de format %c, il entraîne la prise en compte du caractère désigné par le pointeur (*même un séparateur*) et il est avancé sur le caractère suivant.

Les possibilités de la fonction scanf

c. Les premières règles utilisées par scanf :

Exemples : (*n et p sont de type int, c char, @ désigne une fin de ligne*)

scanf("%d%d" , &n, &p);

12^25@ n = 12 p=25

^12^^25^^@ n = 12 p=25

12@

@

^25 n = 12 p=25

scanf("%d%c" , &n, &c);

12^a@ n = 12 c=' '

Les possibilités de la fonction scanf

d. On peut imposer un gabarit maximal :

Le traitement d'un code de format s'interrompt soit à la rencontre d'un séparateur, soit lorsque le nombre de caractère indiqué a été atteint !!

Exemples : (n et p sont de type int, @ désigne une fin de ligne)

```
scanf("%3d%3d" , &n, &p );
```

```
12^25@           n = 12    p=25
```

```
^^^12345@        n = 123   p=45
```

```
12@
```

```
25@              n = 12    p=25
```

Les possibilités de la fonction scanf

e. Rôle d'un espace dans le format :

Un espace entre deux codes de format demande à scanf de faire avancer le pointeur au prochain caractère différent d'un séparateur !!

Exemples : (*n et p sont de type int, c char, @ désigne une fin de ligne*)

```
scanf("%d %c" , &n, &c );
```

```
12^a@          n = 12    c='a'
```

```
12^^^@
```

```
a@             n = 12    c='a'
```

Les possibilités de la fonction scanf

f. Arrêt prématuré de scanf :

Compte = scanf("%d %d %c" , &n, &p, &c);

12^25^b@	n = 12	p = 25	c='b'	compte = 3
12b@	n = 12	p inchangé	c inchangé	compte = 1
b@	n indéfini	p inchangé	c inchangé	compte = 0

Un arrêt prématuré de scanf a lieu dans le cas où scanf n'est pas en mesure de fabriquer une valeur adéquate !!

Les possibilités de la fonction scanf

Le Problème de synchronisation entre l'écran et le clavier :

Examinons cet exemple:

```
#include <stdio.h>
main()
{   int n, p;
    printf("donner une valeur pour n : ");
    scanf("%d",&n);
    printf("Merci pour %d\n",n);
    printf("donner une valeur pour p : ");
    scanf("%d",&p);
    printf("Merci pour %d\n",p);
}
```

```
donnez une valeur pour n : 12 25
merci pour 12
donnez une valeur pour p : merci pour 25
```

Les possibilités de la fonction scanf

f. Problème de synchronisation entre l'écran et le clavier :

Le tampon n'est pas vide à chaque nouvel appel de scanf

g. Erreurs de programmation :

1- Code de format en désaccord avec le type de l'expression:

Si même taille → introduction d'une mauvaise valeur

Si la variable a une taille inférieure → écrasement d'un emplacement mémoire consécutif à cette variable

2- Nombre de codes de format différent du nombre d'éléments de la liste:

Scanf("%d",&n, &p); → seule la valeur de n est lue

Scanf("%d%d",&n); → on affecte une valeur à un emplacement aléatoire de la mémoire

Remarque

- Remarque1

L'expression : `putchar (c)`

joue le même rôle que :

`printf ("%c", c)`

Son exécution est toutefois plus rapide, dans la mesure où elle ne fait pas appel au mécanisme d'analyse de format.

- Remarque2

L'expression : `c = getchar()`

joue le même rôle que :

`scanf ("%c", &c)`

tout en étant plus rapide puisque ne faisant pas appel au mécanisme d'analyse d'un format. Notez bien que `getchar` **utilise le même tampon (image d'une ligne) que `scanf`.**